

NEW UPPER AND LOWER BOUNDS ON COVERING CODES $K_q(n, R)$ FOR ALPHABETS OF SIZE $5 \leq q \leq 21$

MÁRK MAROSI

ABSTRACT. Let $K_q(n, R)$ denote the minimum cardinality of a q -ary code of length n with covering radius R . We improve the known bounds on $K_q(n, R)$ in 84 cases (83 distinct cells). Twenty-six upper bounds for $5 \leq q \leq 15$ are established by explicit codes, found by local search and by a large neighbourhood search whose evaluations are exact coverage counts over the whole space $\mathbb{Z}_q^n$, computed by a coordinate-wise transform; two examples are $K_7(9, 5) \leq 240$ (previously 323) and $K_8(10, 5) \leq 1883$ (previously 2461). Fifty-eight lower bounds for $6 \leq q \leq 21$ are obtained from the semidefinite programming bound of Gijswijt and Polak, whose published computations cover $q \leq 5$, by solving the symmetry-reduced program in multiprecision arithmetic and rounding each dual solution to an exact rational certificate; every certificate is checked by a standalone program in exact arithmetic. The cell $K_6(10, 4)$ is improved from both sides, from 417–2952 to 441–2751. The codes, the certificates, and the checkers are provided as ancillary files.

1. INTRODUCTION

Let $\mathbb{Z}_q^n$ denote the Hamming space of q -ary words of length n . A code $C \subseteq \mathbb{Z}_q^n$ has *covering radius* at most R if every word of $\mathbb{Z}_q^n$ is within Hamming distance R of some codeword, and

$$K_q(n, R) = \min\{|C| : C \subseteq \mathbb{Z}_q^n \text{ has covering radius } \leq R\}.$$

The binary case $K_2(n, 1)$ is the domination number of the hypercube, and $K_3(n, 1)$ is the football pool problem; see [3] for the general theory.

Upper bounds on $K_q(n, R)$ are established by explicit codes and lower bounds by counting or convex-relaxation arguments. The known bounds for $q \geq 3$ are collected in the tables of Kéri [7], which extend those of [3] and were last revised in November 2011; the tables attribute 96 of their upper bounds for $q \geq 6$ to [8]. Recent lower-bound work concerns small alphabets: Wu and Chen [17] improved binary lower bounds, and Gijswijt and Polak [5] obtained new lower bounds for $q \leq 5$ from a semidefinite programming hierarchy. Florath [4] formalizes covering-code bounds in a proof assistant. For $q \geq 6$ we know of no published improvement on either side since 2011, and of none on the upper-bound side for $q \geq 5$; a search of arXiv, DBLP, OpenAlex, Lobstein’s covering-radius bibliography [11], and the publication lists of the authors active in the area, carried out in August 2026, found none.

The contributions of this paper are as follows.

- (1) Twenty-six new upper bounds for $5 \leq q \leq 15$ (Table 1), each established by an explicit code. The codes were found by a local search (Section 4), effective for $q^n \lesssim 10^8$, and by

Date: September 2026.

2020 Mathematics Subject Classification. 94B75 (primary), 05B40, 90C22, 90C59.

Key words and phrases. Covering codes, football pool problem, semidefinite programming, local search, large neighbourhood search.

Department of Artificial Intelligence and Systems Engineering, Budapest University of Technology and Economics (BME), Budapest, Hungary. Email: marosi@mit.bme.hu.

a large neighbourhood search (Section 5) whose evaluations are exact coverage counts over all of $\mathbb{Z}_q^n$, which extends the searchable range to $q^n = 10^{10}$.

- (2) Fifty-eight new lower bounds for $6 \leq q \leq 21$ (Table 2), obtained by solving the symmetry-reduced semidefinite program of [5] in multiprecision arithmetic and rounding each numerical dual solution to an exact rational certificate (Section 6). Each certificate is checked by a standalone program that rebuilds the program from (q, n, R) and verifies the certificate in exact arithmetic.
- (3) Every code was verified by four methods, and every certificate by the exact checker, from the published files (Section 7). The codes, the certificates, and both checkers are ancillary files of this paper.

Section 2 fixes notation, Section 3 states the results, Sections 4 and 5 describe the two upper-bound searches, Section 6 the lower-bound computation, and Section 7 the verification. Implementation measurements are collected in Appendix A, the certified program is stated in Appendix B, and the code establishing $K_6(8, 4) \leq 166$ is printed in Appendix C.

2. PRELIMINARIES

Throughout, $q \geq 2$ and $\mathbb{Z}_q = \{0, \dots, q-1\}$; $d(x, y)$ denotes Hamming distance on $\mathbb{Z}_q^n$, $B_R(x) = \{y : d(x, y) \leq R\}$ the Hamming ball, and $|B_R| = \sum_{i \leq R} \binom{n}{i} (q-1)^i$ its volume. Counting gives the *sphere-covering bound* $K_q(n, R) \geq \lceil q^n / |B_R| \rceil$.

The constructions that propagate upper bounds through the tables of [7] are the *direct sum*

$$(1) \quad K_q(n_1 + n_2, R_1 + R_2) \leq K_q(n_1, R_1) \cdot K_q(n_2, R_2),$$

the inequalities $K_q(n+1, R) \leq q K_q(n, R)$ and $K_q(n+1, R+1) \leq K_q(n, R)$, and product rules over factorizations of the alphabet; see [3, 7] for these and for the source annotations (“keys”) of [7] reproduced in the tables below. For a code C we write $\text{cnt}(w) = \#\{c \in C : d(w, c) \leq R\}$ for the number of codewords covering a word w ; C has covering radius at most R if and only if $\text{cnt}(w) > 0$ for every w .

3. THE NEW BOUNDS

Tables 1 and 2 list the new bounds and Figure 1 shows both sides. The cell $K_6(10, 4)$ is improved from both sides, from 417–2952 to 441–2751, and is the only cell appearing in both tables.

Upper bounds. For 24 of the 26 bounds of Table 1, a search for a code of size $M-1$ with the final computational budget found none. For the two bounds marked $\downarrow$ the search was still producing smaller codes when this version was prepared. The largest relative improvements occur on cells whose previous bound was inherited from a general construction (keys **c**, **d**, **e**, **f**, **j**); the cells whose previous bound was obtained by search in [8] or recorded in [7] (keys **n**, **q**, **z**) improve by between 2.2% and 12.2%. The two searches interact through the tables: the new $K_7(8, 4) \leq 316$ gives $K_7(9, 5) \leq 316$ by monotonicity, and the search in that cell, started from the propagated code, reached $K_7(9, 5) \leq 240$. The closure of all new bounds under the rules of Section 2 gives no further improvement beyond the observation in Remark 1.

Lower bounds. All 58 improved cells carry Kéri keys **y**, **x**, **s** or **a**, that is, their previous lower bound was the sphere-covering bound or a counting refinement of it. The entire family $(n, R) = (8, 2)$, $6 \leq q \leq 21$, is improved, by increments from +91 at $q = 6$ to +14436 at $q = 21$; the largest single increment is $K_{10}(10, 2) \geq 2699348$, previously 2676660. No cell whose previous lower bound is due to Haas, Halupczok and Schlage-Puchta [6] (key **m**) is improved; Section 6.4 reports the values obtained on such cells.

cell	previous bounds	sphere	new UB	Δ	$\Delta\%$	key	via
$K_5(11, 5)$	103–625	86	602	−23	3.7%	d	L
$K_6(7, 3)$	70–246	57	227	−19	7.7%	e	L
$K_6(8, 3)$	246–1080	217	1005	−75	6.9%	f	L
$K_6(8, 4)$	46–216	33	166	−50	23.1%	e	L
$K_6(9, 4)$	136–738	112	660	−78	10.6%	f	L
$K_6(9, 5)$	32–144	21	119	−25	17.4%	j	L
$K_6(10, 4)$	441 –2952	411	2751	−201	6.8%	f	L
$K_6(10, 5)$	83–615	65	484	−131	21.3%	f	L
$K_7(8, 4)$	76–343	56	316	−27	7.9%	c	L
$K_7(9, 4)$	264–1843	221	1475	−368	20.0%	f	L
$K_7(9, 5)$	52–323	35	240	−83	25.7%	f	L
$K_7(10, 5)$	160–1225	126	980	−245	20.0%	f	T
$K_8(6, 4)$	15–20	7	19	−1	5.0%	n	L
$K_8(8, 4)$	118–512	89	505	−7	1.4%	c	L
$K_8(8, 5)$	29–90	15	79	−11	12.2%	z	L
$K_8(10, 5)$	287–2461	225	1883	−578	23.5%	f	T
$K_8(10, 6)$	58–342	37	324 [↓]	−18	5.3%	n	L
$K_9(9, 5)$	120–729	83	684	−45	6.2%	c	T
$K_9(10, 5)$	481–3969	380	3393	−576	14.5%	f	T
$K_{10}(9, 5)$	174–1088	121	1055	−33	3.0%	f	T
$K_{10}(10, 5)$	632–7106	612	5799	−1307	18.4%	f	T
$K_{10}(10, 6)$	122–826	79	802 [↓]	−24	2.9%	n	T
$K_{12}(6, 4)$	30–41	13	39	−2	4.9%	q	L
$K_{13}(6, 4)$	35–46	14	45	−1	2.2%	q	L
$K_{14}(6, 4)$	40–52	16	50	−2	3.8%	q	L
$K_{15}(6, 4)$	46–59	18	57	−2	3.4%	c	L

TABLE 1. New upper bounds. “Previous bounds” are the lower and upper bounds of [7]; a bold lower bound is from Table 2. “Sphere” is $\lceil q^n/|B_R| \rceil$. “Key” is the source annotation of the previous upper bound in [7]: c = monotonicity $K_q(n+1, R+1) \leq K_q(n, R)$; d = Bhandari–Durairajan [1]; e = $K_q(n+1, R) \leq q K_q(n, R)$; f = direct sum; j = alphabet product rule; n = Kéri–Östergård [8]; q = Rivas Soriano (attributed in [7]); z = Kéri 2009 updates recorded in [7]. “Via” is the method that found the code: L = the local search of Section 4, T = the large neighbourhood search of Section 5. Percentages are rounded to one decimal. For the two bounds marked [↓] the search was still producing smaller codes when this version was prepared; for every other bound a search at $M - 1$ with the final budget found no code.

Remark 1. The tabulated bound $K_{17}(7, 2) \leq 252735$ of [7] is weaker than the bound $17 \cdot K_{17}(6, 2) \leq 17 \cdot 14424 = 245208$ that follows from the tabulated $K_{17}(6, 2) \leq 14424$ and the inequality $K_q(n+1, R) \leq q K_q(n, R)$. This involves no new construction.

4. UPPER BOUNDS BY LOCAL SEARCH

4.1. The search. For fixed (q, n, R, M) the search looks for a code of size M minimizing the number of uncovered words. The state is a multiset of M codewords together with $\text{cnt}(w)$ for every $w \in \mathbb{Z}_q^n$; a move changes one coordinate of one codeword.

cell	sphere	prev. LB	new LB	cell	sphere	prev. LB	new LB
$K_6(8, 2)$	2267	2276 (y)	2367	$K_{13}(6, 2)$	2162	2169 (x)	2233
$K_6(9, 2)$	10653	10900 (y)	10965	$K_{13}(7, 2)$	20183	20189 (x)	20545
$K_6(9, 3)$	881	921 (y)	926	$K_{13}(8, 2)$	197562	197563 (x)	199633
$K_6(10, 3)$	3739	3815 (y)	3836	$K_{13}(8, 3)$	8085	8086 (x)	8193
$K_6(10, 4)$	411	417 (s)	441	$K_{14}(6, 2)$	2881	2955 (y)	2964
$K_7(7, 2)$	1031	1035 (x)	1081	$K_{14}(7, 2)$	28952	29273 (y)	29404
$K_7(8, 2)$	5454	5457 (x)	5631	$K_{14}(8, 2)$	305105	305294 (y)	307909
$K_7(8, 3)$	439	457 (y)	471	$K_{15}(6, 2)$	3766	3812 (y)	3862
$K_7(9, 2)$	29870	29889 (s)	30562	$K_{15}(8, 2)$	457578	457584 (x)	461294
$K_7(9, 3)$	2070	2077 (y)	2143	$K_{16}(6, 2)$	4841	4848 (x)	4951
$K_7(10, 2)$	168041	168042 (x)	170632	$K_{16}(7, 2)$	55566	55600 (y)	56237
$K_8(8, 2)$	11741	11766 (y)	12033	$K_{16}(8, 2)$	668894	669207 (y)	673723
$K_8(8, 3)$	813	829 (y)	856	$K_{17}(7, 2)$	74757	75429 (y)	75559
$K_8(9, 4)$	403	409 (s)	429	$K_{17}(8, 2)$	955977	955978 (x)	962145
$K_9(8, 2)$	23181	23184 (x)	23642	$K_{17}(8, 3)$	29475	29478 (x)	29652
$K_9(8, 3)$	1411	1413 (x)	1464	$K_{17}(8, 4)$	1446	1458 (y)	1464
$K_9(9, 3)$	8537	8544 (y)	8685	$K_{18}(6, 2)$	7664	7741 (y)	7804
$K_9(9, 4)$	690	703 (s)	722	$K_{18}(8, 2)$	1339162	1339650 (y)	1346931
$K_9(10, 3)$	54142	54144 (x)	54600	$K_{19}(6, 2)$	9468	9479 (x)	9624
$K_{10}(7, 2)$	5666	5676 (y)	5824	$K_{19}(7, 2)$	128968	128972 (x)	130081
$K_{10}(8, 2)$	42717	42772 (y)	43423	$K_{19}(7, 3)$	4236	4237 (x)	4282
$K_{10}(9, 2)$	333556	333560 (x)	337394	$K_{19}(8, 2)$	1842635	1842639 (x)	1852296
$K_{10}(9, 4)$	1123	1130 (s)	1160	$K_{20}(7, 2)$	165911	167165 (y)	167204
$K_{10}(10, 2)$	2676660	2676660 (a)	2699348	$K_{20}(7, 3)$	5166	5174 (x)	5215
$K_{10}(10, 4)$	6808	6886 (y)	6915	$K_{20}(8, 2)$	2494884	2495614 (y)	2506759
$K_{11}(7, 2)$	8977	9106 (y)	9193	$K_{21}(6, 2)$	14012	14131 (y)	14201
$K_{11}(8, 2)$	74405	74415 (s)	75448	$K_{21}(7, 3)$	6243	6249 (x)	6294
$K_{12}(8, 2)$	123665	123772 (y)	125156	$K_{21}(8, 2)$	3329184	3329193 (x)	3343629
$K_{12}(8, 3)$	5512	5577 (y)	5605	$K_{21}(8, 3)$	82338	82341 (x)	82595

TABLE 2. New lower bounds, each certified by an exact rational dual solution of the program of Appendix B. “Sphere” is $\lceil q^n/|B_R| \rceil$; “prev. LB” is the value of [7] with its key: y = van Wee-type excess counting [16] (our reading of the legend of [7]; see also [3, Ch. 6]), x = Chen and Honkala [2], s = Lang, Quistorff and Schneider [9, 10], a = sphere-covering bound. The exact rational value of the semidefinite bound (a bound on $K_q(n, R)^3$) is stored in each certificate.

Lemma 2. *Let c' be obtained from c by setting coordinate p to $v \neq c_p$. The set of words leaving the radius- R ball of c and the set of words entering the ball of c' are both in bijection with the words at distance exactly R from c in the coordinates other than p . Consequently a move updates exactly $2S$ counters, $S = \binom{n-1}{R}(q-1)^R$, the two sets are disjoint, and distinct updates address distinct words.*

Proof. Since $d(w, c') = d(w, c) + [w_p = c_p] - [w_p = v]$, a word leaves the ball of c if and only if $d(w, c) = R$ and $w_p = c_p$, and enters the ball of c' if and only if $d(w, c') = R$ and $w_p = v$. Writing such a word as its restriction to the coordinates other than p (at distance exactly R from the restriction of c) together with its value at p gives the bijections, disjointness, and injectivity. (For $R \geq n$ both sets are empty; all cells considered here have $R < n$.) $\square$

The search is a focused local search of WalkSAT type, related to the simulated annealing and tabu searches used to construct many entries of the tables [15, 13]: select a random uncovered word u ; take as candidate moves every codeword at distance exactly $R + 1$ from u moved one

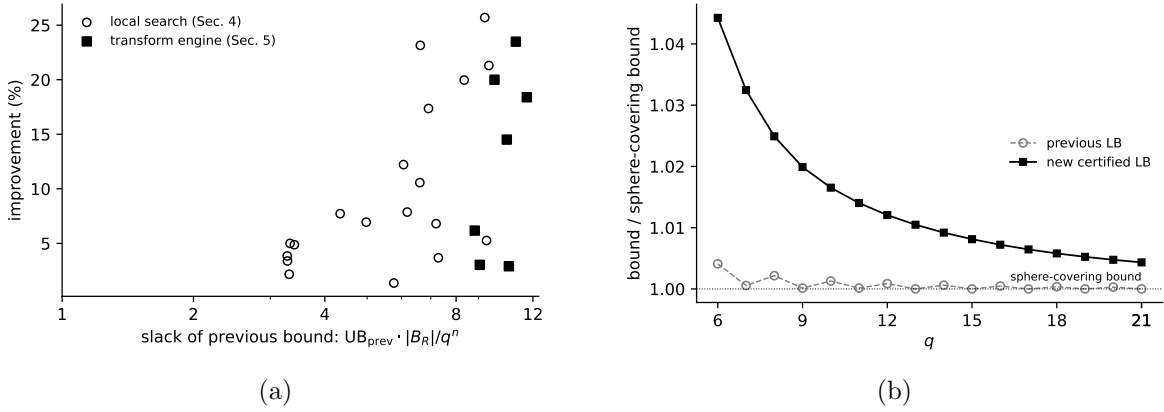


FIGURE 1. (a) Relative improvement of each upper bound of Table 1 against the ratio of the previous bound to the sphere-covering bound; squares mark the cells found by the large neighbourhood search of Section 5. (b) The lower bounds for $(n, R) = (8, 2)$, $6 \leq q \leq 21$: previous bound, sphere-covering bound, and the certified semidefinite bound of Table 2.

coordinate towards u (if no codeword is that close, the codewords at minimum distance from u); evaluate all candidates exactly; apply a best candidate, ties broken uniformly at random; when no progress has been made for a fixed number of iterations, move one codeword onto an uncovered word.

The initial code for a cell is rebuilt from the construction that gave the tabulated bound (for $K_6(10, 4)$, the direct sum (1) of an optimal $K_6(4, 1)$ code and a $K_6(6, 3)$ code of the tabulated size, both found by search and verified). When a code of size M is found, the codeword that is the sole coverer of the fewest words is deleted and the search continues at $M - 1$.

4.2. Evaluation of candidate moves. Candidate evaluation takes at least 92.9% of the run time on the reference cells of Appendix A.1. The following reformulation computes the same quantities with less work.

Lemma 3. *With $D(w) = \{j : w_j \neq c_j\}$, a word w leaves the ball of c under the move at coordinate p if and only if $|D(w)| = R$ and $p \notin D(w)$; it enters the ball of the moved codeword if and only if $|D(w)| = R + 1$, $p \in D(w)$ and $w_p = v$.*

Proof. $|D(w)| = d(w, c)$, and the move changes the disagreement set only at coordinate p ; apply the distance identity in the proof of Lemma 2. $\square$

Let $T = \#\{w : |D(w)| = R, \text{cnt}(w) = 1\}$ and $H_j = \#\{w : |D(w)| = R, \text{cnt}(w) = 1, j \in D(w)\}$. By Lemma 3 the number of words uncovered by the move at p equals $T - H_p$, for every p simultaneously: a leaving word with $\text{cnt}(w) = 1$ is covered by c alone, and it is not re-covered by the moved codeword since $w_p = c_p \neq v$. One enumeration of the distance- R sphere of c therefore gives the loss of all $n(q - 1)$ single-coordinate moves of c . The gain of a move depends on the new value v and concerns only words with $\text{cnt}(w) = 0$, so it is obtained from an explicitly maintained list U of uncovered words in $O(n|U|)$ operations. Evaluation reads only whether a counter is 0, 1 or larger, so a two-bit array $\min(\text{cnt}(w), 2)$ serves the reads; on $K_8(9, 4)$ this reduces the memory read during evaluation from 268 MB to 33.5 MB.

Each reformulation computes the same objective differences as the reference implementation and therefore follows the same search trajectory from the same seed; this was checked on 24

configurations with $2 \leq q \leq 7$, including $R = 0$ and $R = n - 1$. The resulting speedup at identical trajectory is 2–10 times on the reference cells (Appendix A.1). Evaluating all $n(q - 1)$ moves of the candidate codewords instead of the n moves towards u changes the search; it reduces the time to a fixed target on the two larger reference cells and increases it on $K_6(6, 3)$ (Table 4).

4.3. Structured codes. Two restricted searches complement the search over arbitrary subsets of $\mathbb{Z}_q^n$.

Unions of cosets of a linear code. When q is a prime power and $M = cq^k$, we search codes of the form $C = \bigcup_{i=1}^c (x_i + L)$ with L a linear $[n, k]_q$ code. Such a C has covering radius at most R if and only if the syndromes of $x_1, \dots, x_c$ cover the q^{n-k} syndromes under the coset-leader weights of L , a problem on q^{n-k} rather than q^n elements. This search reproduces the tabulated $K_8(8, 4) \leq 512 = 8^3$ and $K_5(11, 5) \leq 625 = 5^4$ in under a second, whereas the unrestricted search did not reach these sizes; the searches leading to 505 and 602 start from these codes.

Codes invariant under translation by 1. A second search works with codes of the form $C = C_0 \cup C_1$, where C_0 is invariant under $x \mapsto x + \mathbf{1}$ and is represented in the quotient of size q^{n-1} , and C_1 is an arbitrary set of codewords. On $K_6(8, 4)$, neither a fully invariant code nor the unrestricted search reached the target size in our runs, while $C_0 \cup C_1$ with a small C_1 did so repeatedly.

4.4. Configuration. The search program combines four implementations: the search of Section 4.2, a SIMD variant of it, and the two restricted searches of Section 4.3. The combination was chosen by comparing the four on a fixed set of instances with held-out evaluation instances and scoring by verified output only (Appendix A.2); no single implementation performed best on all instances. A run first executes the coset search over all feasible k and builds direct sums over all splits of n and R from recorded codes, then runs a set of independent search processes whose composition depends on q^n and on $q^n/(M|B_R|)$; the final code is re-read from disk, duplicates are removed, and coverage is recomputed by marking balls.

The nineteen codes of Table 1 marked L were found by this program on 64 CPU cores, with between seconds and a few core-days per cell; the seven codes marked T are the subject of Section 5.

5. UPPER BOUNDS BY LARGE NEIGHBOURHOOD SEARCH WITH EXACT COVERAGE COUNTS

For $q^n \gtrsim 10^8$ the search of Section 4 makes little progress: the arrays of each process exceed the CPU caches and each move requires a walk over a sphere of S words at memory latency. (The one such cell in Table 1 found by that search is $K_8(10, 6)$, where the radius-6 balls are so large that the uncovered set stays small.) Two ports of the local search to a GPU were implemented and measured (Appendix A.3); their throughput was at most 1.24 times that of the 64-core CPU. The method of this section instead computes exact coverage counts for every word of $\mathbb{Z}_q^n$ at once.

5.1. Distance-count transforms. For $S \subseteq \mathbb{Z}_q^n$ and $0 \leq d \leq R$ let $N_d^S(x) = \#\{u \in S : d(x, u) = d\}$.

Proposition 4. *Let A_0 be the indicator array of S on $\mathbb{Z}_q^n$ and $A_1 = \dots = A_R = 0$. For each axis $j = 1, \dots, n$ and every one-dimensional fiber $\{x + te_j\}_{t \in \mathbb{Z}_q}$, perform, for $d = R, R-1, \dots, 1$ in this order,*

$$A_d[a] \leftarrow A_d[a] + \left(\sum_b A_{d-1}[b] \right) - A_{d-1}[a],$$

where a, b range over the fiber and the sum is formed before any write in that fiber and grade. After all n axes, $A_d = N_d^S$ for $0 \leq d \leq R$. The number of additions is $\Theta(nRq^n)$.

Proof. By induction on the number of processed axes, after axes $1, \dots, j$ the entry $A_d(x)$ counts the $u \in S$ that differ from x in exactly d of the first j coordinates and agree with x in the remaining ones: the update adds, at each position a of the fiber, the number of such u at grade $d - 1$ over the first $j - 1$ axes whose j -th coordinate differs from a . The descending order of d ensures that the update of A_d reads A_{d-1} as left by the previous axis, since A_{d-1} is overwritten only after every A_d that reads it; A_0 is never modified. Forming the fiber sum before writing ensures that no position reads a partially updated neighbour. Each axis touches every entry of $A_1, \dots, A_R$ a bounded number of times, which gives the operation count. $\square$

The transform is the fast product-domain technique of Yates [18] applied to the filtration of $\mathbb{Z}_q^n$ by Hamming distance and truncated at grade R . Its inner loop is branchless and identical on all q^{n-1} fibers, and it is bandwidth-bound on a GPU: on the machine of Appendix A one transform takes 63 ms at $q^n = 7^{10}$ and 143 ms at 8^{10} with 32-bit counters ($4(R + 1) + 2$ bytes per word of $\mathbb{Z}_q^n$).

Two instances of Proposition 4 are used. With S the set of uncovered words, $g(x) = \sum_{d \leq R} N_d^S(x)$ is the number of words that a codeword placed at x would newly cover, for every $x \in \mathbb{Z}_q^n$; with $\bar{S}$ the set of words with $\text{cnt}(w) = 1$, $\ell(c) = \sum_{d \leq R} N_d^{\bar{S}}(c)$ is the number of words that deleting c would uncover, for every codeword c .

5.2. The search. The search is a large neighbourhood search [14] over these two maps. Starting from a code (a direct sum or a recorded code), it deletes a set of codewords chosen among those with small ℓ , together with some codewords chosen at random, and then rebuilds by repeatedly placing a codeword at a maximizer of g , recomputing or locally correcting g after each placement. When the rebuilt code has covering radius at most R at size M , the same step is applied at $M - 1$. Each placement is optimal with respect to the exact coverage counts, whereas a move of Section 4 changes one coordinate; the search performs far fewer moves. The greedy rebuild also serves as a constructor from the empty code: on $K_8(10, 4)$ ($q^n = 1.07 \cdot 10^9$) it produces a code of size 14946 and covering radius 4 in three minutes, while the search of Section 4 did not produce any covering code of that cell.

Five bounds of Table 1, on cells with $2.8 \cdot 10^8 \leq q^n \leq 3.5 \cdot 10^9$, were found by this search with all arrays in GPU memory, within hours per cell. The remaining two are described next.

5.3. Cells with $q^n = 10^{10}$. With all arrays in the 96 GB of GPU memory the largest cells that can be searched have $q^n \approx 3.5 \cdot 10^9$. For larger cells the arrays $A_1, \dots, A_R$, which are read and written only by the axis passes of Proposition 4, are placed in the 480 GB of CPU memory, which the GPU addresses directly, while the array cnt , which is accessed at random by ball walks, stays in GPU memory. A transform then takes about 24 times longer (Table 5). In this configuration the search found $K_{10}(10, 5) \leq 5799$, previously 7106 (a direct sum), in six hours on one machine, and $K_{10}(10, 6) \leq 802$, previously 826.

5.4. Cells on which the search fails. The tabulated bound $K_8(10, 4) \leq 11776$ is the direct sum of a $K_8(4, 2)$ code of size 23 and a $K_8(6, 2)$ code of size 512. In this code every codeword is the sole coverer of many more words than a codeword of a random code of the same size, so deleting k codewords uncovers a correspondingly large set that a rebuild with $k - 1$ codewords must cover again. Runs of several hours found no code of size 11775, and likewise none below the direct sum $K_9(10, 4) \leq 27 \cdot 729$. A code of size 22 in $K_8(4, 2)$ would give $K_8(10, 4) \leq 11264$; local search, SAT solving (with and without symmetry breaking) and integer programming neither found such a code nor proved that none exists, and $K_8(4, 2) \in \{22, 23\}$ remains open.

6. LOWER BOUNDS

6.1. The semidefinite program. Gijswijt and Polak [5, Theorem 4.18] bound $K_q(n, R)^3$ from below by the optimum of a semidefinite program in variables indexed by the orbits of triples of words under $\text{Aut}(H(n, q)) = S_q \wr S_n$, with positive semidefiniteness constraints from the block diagonalization of the Terwilliger algebra of the Hamming scheme, linear constraints expressing sphere-covering inequalities, and the matrix-cut constraints of their Proposition 4.17. The program is stated in Appendix B. For $q \geq 3$ its size depends only on n (at $n = 11$: 339 variables and 126 blocks of order at most 13), and q enters only through the coefficients; the computations published in [5] cover $q \leq 5$.

We reimplemented the reduced program with the generation of all coefficients in exact integer arithmetic. The implementation was checked as follows: (a) for each length $2 \leq n \leq 10$ an explicit verified code of covering radius R (for $6 \leq n \leq 10$, codes from Table 1) was substituted into the program and every constraint was checked in exact rational arithmetic, with objective value exactly $|C|^3$; (b) every coefficient generated for six cells with $3 \leq q \leq 7$ was compared with the output of the authors' published code, with no difference; (c) of 49 values of the tables of [5] recomputed, 44 agree to the displayed precision, and 18 of the 22 improved lower bounds attempted are reproduced exactly; the four others ($K_4(7, 1)$ by 1, $K_5(7, 1)$ by 7, $K_5(8, 1)$ and $K_5(8, 2)$) were computed in double precision, and $K_5(8, 2) \geq 861$ was subsequently reproduced by the multiprecision computation of Section 6.3; (d) no certified bound exceeds the known upper bound of its cell, checked on all 1145 cells of [7].

6.2. Certificates. A dual feasible point of the program (multipliers $y_k \geq 0$ for the linear constraints and positive semidefinite matrices Y_b for the blocks) proves the bound it attains (Lemma 5). Each numerical dual solution is rounded to rationals with a dyadic denominator, each Y_b is displaced into the interior of the semidefinite cone, and if a dual inequality then fails in exact arithmetic the whole dual is multiplied by the largest dyadic $\theta \leq 1$ that restores feasibility, which multiplies the bound by θ .

The certificate is checked by a standalone program (Python standard library only) that rebuilds the program from (q, n, R) in exact integer arithmetic, verifies $y_k \geq 0$, verifies $Y_b \succeq 0$ by an exact LDL^T factorization over $\mathbb{Q}$, verifies every dual inequality in integer arithmetic, and derives the integer bound by exact comparison of cubes. The checker also verifies the validity of the covering inequality named in the certificate. The checker and the solving pipeline share the transcription of the program of [5] into coefficient generators; an error in that transcription would not be detected by a certificate, which is why the transcription is checked separately by the four tests above. No floating-point computation enters the checker.

6.3. Numerical solution. The optimum $|C|^3$ can exceed 10^{13} while the constraint constants are $O(1)$ and the objective coefficients span ten orders of magnitude. All computations therefore use a scaling by powers of two: the objective is divided by the cube of the sphere-covering bound and each variable is scaled by the square root of its objective coefficient, rounded to a power of two, so that the inverse map to the unscaled dual is exact. On $K_6(8, 3)$ in double precision this scaling raises the certified bound from 183 to 240; the cube root of the optimum is about 239.52.

Double-precision interior-point solvers fail on this program beyond optima of about $1.5 \cdot 10^9$ (cube root about 1150). All 58 certificates of Table 2 were therefore obtained with SDPA-GMP [12] at 200 or 250 bits of working precision. The problem files carry exact dyadic coefficients, the multiprecision dual is parsed into exact rationals, and rounding is performed in the scaled coordinates with denominator 2^{128} (one certificate: 2^{64}), with the displacement into the cone proportional to the rounding step. Of the 58 certificates, 52 have $\theta = 1$ and the

other six have $\theta \geq 1 - 2.3 \cdot 10^{-4}$; in no case does the scaling change the integer bound. The pipeline reproduces the 512-bit value $K_5(8, 2) \geq 861$ of [5] exactly.

6.4. Cells that are not improved. After the cube root, the semidefinite bound exceeds the sphere-covering bound by a factor that decreases with q (for $(n, R) = (8, 4)$: 1.44 at $q = 4$ and 1.16 at $q = 7$). It therefore improves the tabulated bounds on cells whose previous bound is close to the sphere-covering bound, which are the cells with keys **y**, **x**, **s** and **a**, and the whole family $(n, R) = (8, 2)$, $6 \leq q \leq 21$ (Figure 1(b)). On the cells attempted whose previous bound is due to [6] (key **m**, between 11% and 67% above the sphere-covering bound), the certified value is at most the tabulated one. For $R = 1$ and $q \geq 6$ the bound is close to the sphere-covering bound: for $K_6(7, 1)$ the certified value equals $6^7/36 = 7776$, and for $K_6(8, 1)$ it exceeds the sphere-covering bound by 1.5% and is below the tabulated 41991.

A certificate whose value does not exceed the tabulated bound is not an improvement, and a certificate whose value is below the sphere-covering bound, which occurs when the solver does not converge, carries no information. Of the 196 certificate files produced in this work, 58 improve the tabulated bound, 28 attain it exactly, 82 certify a value between the sphere-covering bound and the tabulated bound, and 28 certify a value below the sphere-covering bound. Only the 58 improving certificates appear in Table 2 and in the ancillary files.

The fractional covering linear program does not improve any cell: every word of $\mathbb{Z}_q^n$ lies in exactly $|B_R|$ balls, so summing all covering constraints gives LP optimum at least $q^n/|B_R|$, and the uniform solution $1/|B_R|$ attains it.

7. VERIFICATION

Upper bounds. Each code, read from its published file, was checked by four methods, two of which share no code with the other two: (i) marking the balls of all codewords in a q^n -bit array; (ii) meet-in-the-middle covering with bitsets over a split of the coordinates; (iii) computing the minimum distance from every word of $\mathbb{Z}_q^n$ to the code, which also gives the exact covering radius; (iv) R -fold dilation of the indicator array of the code along the coordinate axes, which computes no distances. A fifth dilation check, written separately from the four and included in the repository’s build script, re-verified all 26 codes from the published files before this version was prepared. The number of distinct codewords in each file is compared with the claimed M . For the largest cells the check is itself a computation: the dilation of $K_9(10, 5)$ takes a few GB of memory and minutes, that of $K_{10}(10, 5)$ tens of GB or the meet-in-the-middle method; the ancillary README lists the resource needs.

Lower bounds. All 196 certificates were re-checked by the exact checker before this version was prepared, with the outcome stated in Section 6.4; none certifies a value exceeding a known upper bound.

The ancillary files contain the 26 codes (one codeword per line, digits 0–9 then **a–z** for $q > 10$), the 58 certificates of Table 2, and the checkers `verify_cov.py`, `verify_independent.py` and `certify.py`.

Data availability. The codes, the certificates, the checkers, and a README are ancillary files of the arXiv version of this paper. The search, certification, and verification source code, together with a machine-readable table merging the bounds of [7] with the post-2011 literature and the present improvements, is available at <https://github.com/Mapika/coldcase>.

Use of artificial intelligence. The manuscript was drafted and edited, and the search, certification, and verification software written, by the AI system Claude (Anthropic) under

the direction of the author. The author has verified the statements and proofs in this paper and takes full responsibility for its content.

REFERENCES

1. Mahesh C. Bhandari and C. Durairajan, *A note on bounds for q -ary covering codes*, IEEE Transactions on Information Theory **42** (1996), no. 5, 1640–1642, doi:10.1109/18.532916.
2. Wende Chen and Iiro S. Honkala, *Lower bounds for q -ary covering codes*, IEEE Transactions on Information Theory **36** (1990), no. 3, 664–671, doi:10.1109/18.54887.
3. Gérard Cohen, Iiro Honkala, Simon Litsyn, and Antoine Lobstein, *Covering codes*, North-Holland Mathematical Library, vol. 54, North-Holland, Amsterdam, 1997.
4. Andreas Florath, *Formal foundations and proof-carrying certificates for q -ary covering codes in Lean 4*, 2026, arXiv:2606.09600.
5. Dion Gijswijt and Sven Polak, *Semidefinite lower bounds for covering codes*, 2025, arXiv:2504.01932 (v2, June 2026).
6. Wolfgang Haas, Immanuel Halupczok, and Jan-Christoph Schlage-Puchta, *Lower bounds for q -ary codes with large covering radius*, Electronic Journal of Combinatorics **16** (2009), no. 1, R133, doi:10.37236/222.
7. Gerzson Kéri, *Tables for bounds on covering codes*, <https://old.sztaki.hu/~keri/codes/>, Last updated November 2011; accessed August 2026. A mirror is included in the repository accompanying this paper.
8. Gerzson Kéri and Patric R. J. Östergård, *Bounds for covering codes over large alphabets*, Designs, Codes and Cryptography **37** (2005), 45–60, doi:10.1007/s10623-004-3804-8.
9. W. Lang, Jörn Quistorff, and E. Schneider, *New results on integer programming for codes*, Congressus Numerantium **188** (2007), 97–107.
10. ———, *Integer programming for covering codes*, Journal of Combinatorial Mathematics and Combinatorial Computing **66** (2008), 279–288.
11. Antoine Lobstein, *Covering radius, a bibliography*, <https://www.lri.fr/~lobstein/bib-a-jour.pdf>, 1058 references; last updated January 10, 2023.
12. Maho Nakata, *A numerical evaluation of highly accurate multiple-precision arithmetic version of semidefinite programming solver: SDPA-GMP, -QD and -DD*, Proceedings of the IEEE International Symposium on Computer-Aided Control System Design, 2010, doi:10.1109/CACSD.2010.5612693, pp. 29–34.
13. Patric R. J. Östergård, *Constructing covering codes by tabu search*, Journal of Combinatorial Designs **5** (1997), 71–80, [https://doi.org/10.1002/\(SICI\)1520-6610\(1997\)5:1<71::AID-JCD6>3.0.CO;2-W](https://doi.org/10.1002/(SICI)1520-6610(1997)5:1<71::AID-JCD6>3.0.CO;2-W).
14. Paul Shaw, *Using constraint programming and local search methods to solve vehicle routing problems*, Principles and Practice of Constraint Programming (CP 1998), Lecture Notes in Computer Science, vol. 1520, Springer, 1998, doi:10.1007/3-540-49481-2_30, pp. 417–431.
15. P. J. M. van Laarhoven, E. H. L. Aarts, J. H. van Lint, and L. T. Wille, *New upper bounds for the football pool problem for 6, 7, and 8 matches*, Journal of Combinatorial Theory, Series A **52** (1989), 304–312, doi:10.1016/0097-3165(89)90036-8.
16. Gerhard J. M. van Wee, *Improved sphere bounds on the covering radius of codes*, IEEE Transactions on Information Theory **34** (1988), no. 2, 237–245.
17. Ying-Sian Wu and Jun-Yo Chen, *Improved lower bounds on the domination number of hypercubes and binary codes with covering radius one*, Discrete Mathematics **347** (2024), 113752, doi:10.1016/j.disc.2023.113752.
18. Frank Yates, *The design and analysis of factorial experiments*, Imperial Bureau of Soil Science, Harpenden, 1937.

APPENDIX A. IMPLEMENTATION MEASUREMENTS

All measurements were made on one machine with 64 ARM CPU cores, 480 GB of CPU memory and one GPU with 96 GB of memory, the CPU memory being addressable by the GPU. Comparative figures are process CPU time, or paired measurements in the same time window on the same machine.

A.1. The evaluation of Section 4.2. On three reference cells of increasing size ($K_6(6,3)$ at $M = 41$, $K_6(8,4)$ at $M = 169$, $K_8(9,4)$ at $M = 2944$) candidate evaluation takes 92.9%, 98.5% and 99.6% of the run time, and one evaluated sphere pattern costs from 1.9 ns to 7.4 ns

as the counter array grows from 93 kB to 268 MB. Table 3 gives the speedup at a fixed search trajectory and Table 4 the speedup to a fixed quality target.

variant	$K_6(6, 3)$ 20 000 it	$K_6(8, 4)$ 600 it	$K_8(9, 4)$ 50 it
reference implementation	1.00×	1.00×	1.00×
unrolled walk, precomputed offsets	1.35×	1.29×	1.13×
+ 8-bit state array	1.28×	1.65×	1.65×
+ 2-bit state array	0.82×	1.34×	2.44×
shared sphere, 8-bit state	1.32×	1.95×	1.95×
+ uncovered-word list	2.21×	5.57×	6.06×
with 2-bit state	2.14×	6.32×	9.86×

TABLE 3. Speedup in CPU time at a fixed iteration budget, median of 9 runs (6 on the largest cell). All rows perform the same search: the final uncovered counts agree, seed by seed, across rows. The interquartile ranges are 0.9%, 2.2% and 5.6% of the median.

cell	M	target	identical search	all $n(q-1)$ moves
$K_6(6, 3)$	41	$ U \leq 30$	$2.03 \times [1.99, 2.13]$	$0.28 \times$
$K_6(8, 4)$	169	$ U \leq 20$	$6.12 \times [6.06, 6.62]$	$8.03 \times [5.75, 11.02]$
$K_8(9, 4)$	2944	$ U \leq 10\,800$	$8.07 \times [6.19, 10.72]$	$11.04 \times [8.06, 12.17]$

TABLE 4. Median per-seed speedup in CPU time to the target over 20 seeds, with percentile-bootstrap 95% confidence intervals. In the fourth column the optimized program reached the target at the same iteration as the reference on every seed. The fifth column also evaluates all $n(q-1)$ moves of each candidate codeword, which changes the search.

The following modifications were tested and not adopted. Caching move evaluations between iterations requires the evaluation spheres of successive moves to be disjoint, that is $d(c, c') > 2R + 2$; since $2R + 2 \geq n$ on every cell considered, every move invalidates every cached value. Maintaining the list of words with $\text{cnt}(w) = 1$ in the same way as U pays only if $n|U_1| < \binom{n}{R}(q-1)^R$, which fails by one to two orders of magnitude. Fixing one codeword at 0^n gave no measurable change. Choosing as focus the word uncovered for the longest time instead of a random uncovered word performed much worse. Restarts reduced the time to a hard target on $K_6(6, 3)$ (speedup $4.2 \times$, $p = 0.002$) and had no effect on the cells of Table 1 (coefficient of variation of the time to target 0.20–0.47; $p = 1.0$).

A.2. Choice of the configuration of Section 4.4. The four implementations were compared under one protocol: identical interface and resource limits, a set of development instances, a set of held-out evaluation instances, 15 seeds per instance, and a score computed from verified output only (a verified covering code scores 1000, a partial cover scores minus its uncovered fraction, and invalid output scores below every valid partial cover). The restricted searches scored highest on the held-out instances, the search of Section 4.2 was fastest on large cells, the SIMD variant on cache-resident cells, and single-threaded processes scored higher than multi-threaded ones except on the largest cells. Under the same protocol the combined program solved 14 of 20 held-out instances, against 12 of 20 for the best single implementation.

A.3. GPU measurements for Section 5. *Ports of the local search.* A formulation of candidate evaluation as integer matrix products has contracted dimension $nq \approx 50$ and did not reach CPU throughput. A port running 256 independent search processes in one persistent kernel reached 0.09, 0.10 and 1.24 times the throughput of the 64 CPU cores on cells with $q^n = 4.7 \cdot 10^4$, $1.7 \cdot 10^6$ and $1.3 \cdot 10^8$.

Memory placement for $q^n > 3.5 \cdot 10^9$ (Section 5.3). The GPU addresses the CPU memory through a cache-coherent interconnect with native atomic operations. The arrays $A_1, \dots, A_R$ are placed in CPU memory and cnt in GPU memory (for $K_{10}(10, 5)$: 240 GB and 21 GB). The placement must be fixed explicitly: allocation through the unified-memory allocator is about 30 times slower than `malloc` with direct access through the coherent page tables, and with plain `malloc` the operating system migrates pages touched by the GPU into GPU memory until it is full; binding the allocations to the CPU memory node with `mbind` makes the placement deterministic. The outputs were identical across all placements. Table 5 gives the time of one transform.

cell	q^n	arrays	placement	transform
$K_8(10, 4)$	$1.1 \cdot 10^9$	24 GB	GPU memory	0.143 s
$K_8(10, 4)$	$1.1 \cdot 10^9$	24 GB	CPU memory (bound)	3.4 s
$K_8(10, 4)$	$1.1 \cdot 10^9$	24 GB	CPU memory (managed)	8.7 s
$K_9(10, 5)$	$3.5 \cdot 10^9$	91 GB	CPU memory (bound)	22–26 s
$K_{10}(10, 5)$	10^{10}	260 GB	CPU memory (bound)	73–110 s

TABLE 5. Time of one transform of Proposition 4 by placement of $A_1, \dots, A_R$, on an idle machine; cnt is in GPU memory in all rows.

Ball walks. Ball walks use a precomputed table of packed coordinate-shift patterns, so that each visited word costs one table load and at most R additions in shared memory, without division or modulo operations. Splitting each walk over up to 8192 thread blocks (instead of one) makes single-word updates 30 times faster on $K_8(9, 4)$ ($0.80 \rightarrow 0.027$ ms) and 37 times faster on $K_8(10, 4)$ ($1.34 \rightarrow 0.036$ ms), batched candidate evaluations 3.7–4.3 times faster, and a complete greedy construction on $K_8(9, 4)$ 1.4 times faster at identical output. Recording each word’s last coverer during the marking pass, so that all $\ell(c)$ are obtained from one linear scan, replaces one transform but is slower (28 vs. 17 ms on $K_8(9, 4)$, 669 vs. 145 ms on $K_8(10, 4)$), since the marking pass performs $M \cdot |B_R|$ atomic operations while the transform streams $O(nRq^n)$ words.

A.4. Baselines. *Integer programming.* The set-covering integer program with one binary variable and one constraint per word of $\mathbb{Z}_q^n$ has $q^n \cdot |B_R|$ nonzero coefficients. Among the improved cells with $q^n \leq 10^8$ only $K_6(7, 3)$ ($1.4 \cdot 10^9$ nonzeros, about 17 GB) fits in 480 GB of memory; the other $n = 6$ cells need between 137 GB and 7.2 TB for the constraint matrix. On $K_6(7, 3)$, HiGHS with 8 threads and one hour per run produced neither a feasible cover nor a dual bound above the sphere-covering bound, in minimization mode and in feasibility mode at sizes 245 and 227.

Plain tabu search. A tabu search from the same code base without the modifications of Section 4.2 was run against the program of Section 4.4 with 8 CPU-hours per cell, identical seeds and the same time window:

cell (target M)	[7]	plain tabu	Section 4.4
$K_8(6, 4)$ (19)	20	solved	solved
$K_{12}(6, 4)$ (40, 39)	41	solved, solved	solved, solved
$K_{13}(6, 4)$ (45)	46	solved	solved
$K_{14}(6, 4)$ (51, 50)	52	solved, solved	solved, solved
$K_{15}(6, 4)$ (58, 57)	59	solved, failed	solved, solved
$K_6(7, 3)$ (245, 227)	246	solved, failed	solved, failed

The $n = 6$ improvements other than $K_{15}(6, 4) \leq 57$ are thus reachable by a plain tabu search with this budget. Neither program reached $K_6(7, 3) \leq 227$ from scratch with this budget; that code was obtained by the sequence of searches at decreasing M described in Section 4.1.

APPENDIX B. THE CERTIFIED PROGRAM

The reduced program of [5, Theorem 4.18] (non-binary case, $q \geq 3$) is a minimization in variables $x \in \mathbb{R}^N$ indexed by the orbits of triples of words under $\text{Aut}(H(n, q)) = S_q \wr S_n$:

$$(2) \quad \begin{aligned} \min \quad & c^\top x \quad \text{s.t.} \quad x \geq 0, \quad \langle l_k, x \rangle + l_k^0 \geq 0 \quad (k = 1, \dots, m), \\ & C_b + \sum_{v=1}^N x_v A_b^v \succeq 0 \quad (b = 1, \dots, B), \end{aligned}$$

where the data c , (l_k, l_k^0) , (C_b, A_b^v) are rationals determined by (q, n, R) : the objective and the linear constraints encode sphere-covering inequalities and the matrix-cut inequalities of [5, Proposition 4.17], and the semidefinite blocks come from the block diagonalization of the Terwilliger algebra of the nonbinary Hamming scheme [5, §3]. By [5, Theorem 4.18], every code $C \subseteq \mathbb{Z}_q^n$ of covering radius at most R gives a feasible point x_C of (2) with $c^\top x_C = |C|^3$; hence $K_q(n, R)^3 \geq \text{OPT}$, the optimum of (2).

A certificate for (q, n, R) consists of $y \in \mathbb{Q}_{\geq 0}^m$ and rational symmetric positive semidefinite matrices $Y_1, \dots, Y_B$, stored over one common denominator. Let

$$d_v = \sum_k y_k l_k[v] + \sum_b \langle Y_b, A_b^v \rangle \quad (v = 1, \dots, N), \quad d_0 = \sum_k y_k l_k^0 + \sum_b \langle Y_b, C_b \rangle.$$

Lemma 5. *If $d_v \leq c_v$ for every v , then every feasible point x of (2) satisfies $c^\top x \geq -d_0$; hence $K_q(n, R) \geq \lceil (-d_0)^{1/3} \rceil$.*

Proof. For feasible x ,

$$c^\top x + d_0 = \sum_v (c_v - d_v) x_v + \sum_k y_k (\langle l_k, x \rangle + l_k^0) + \sum_b \left\langle Y_b, C_b + \sum_v x_v A_b^v \right\rangle \geq 0,$$

since each summand is a product of nonnegative quantities ($x \geq 0$ and $c_v - d_v \geq 0$; $y_k \geq 0$ and the linear constraint value ≥ 0 ; the inner product of two positive semidefinite matrices is nonnegative). The bound on $K_q(n, R)$ follows from $K_q(n, R)^3 \geq \text{OPT} \geq -d_0$. $\square$

If the rounded dual has $d_v > c_v$ for some v , the certificate is multiplied by the largest dyadic $\theta \leq 1$ with $\theta d_v \leq c_v$ for all v , which gives the bound $-\theta d_0$. The checker verifies the hypotheses of Lemma 5 (nonnegativity of y , positive semidefiniteness of each Y_b by exact $LDL^\top$ factorization over $\mathbb{Q}$, and the N inequalities $d_v \leq c_v$ in integer arithmetic) and rebuilds the data c , (l_k, l_k^0) , (C_b, A_b^v) from (q, n, R) ; the transcription of that data from [5] is the part of the computation not covered by the certificate, and it is checked as described in Section 6.

APPENDIX C. THE CODE ESTABLISHING $K_6(8, 4) \leq 166$

The code is printed here because it is the largest relative improvement of Table 1 that fits on a page (the codes for $K_7(9, 5) \leq 240$ and $K_8(10, 5) \leq 1883$ are longer). Each row lists four codewords of $\mathbb{Z}_6^8$; the machine-readable version is the ancillary file `K6_8_4_M166.txt`.

01430421	12541532	23052043	30103154
45214205	50325310	03405225	14510330
25021441	30132552	41243003	52354114
03134001	14245112	25350223	30401334
51512445	52010550	02442243	13553304
24004415	35115520	40220031	51331142
02303231	13414342	24525453	35030504
40141015	51252120	05143544	10254055
21305100	32410211	43521322	54032433
04301535	13432040	20543151	31054202
42105313	53210434	04043240	15154351
20205402	31310513	42421024	53532135
05111133	10222244	21333355	32444400
43555511	54000022	01045452	12150503
23201014	34312125	45423230	50534341
00544125	11055230	22100341	33211452
44322503	55433014	02231500	13342011
24453122	35504233	40015344	51120455
05223315	10334420	21445531	32550042
43001153	54112204	03411050	14542101
25033212	30144323	41255434	52300445
04450305	15501410	20012521	31123032
42234143	53345254	00052413	11103524
22214035	33324140	44430251	55541302
02115012	13220123	24331234	35442345
40553450	51004501	01024314	12135425
23240530	34351041	45402152	50513203
05352332	10413443	21514554	52025505
43130110	54241221	04205542	15315053
20420104	31531215	42042320	53153431
01213111	12324222	23435333	34540444
45051555	42131403	00300252	11411303
22522414	33033525	44104030	55255141
05535024	10040135	21151240	32202351
44313402	54424513	01500043	12051134
23122205	34233310	45344421	50455532
03314404	10402515	25530020	31021031
41132242	52243353	12005001	03402520
31455113	23344502	55203541	32515205
30325000	02023150	14021545	54142054
34124521	03523252	45250342	00522424
44553234	53044110		